

DAC Testnet Infrastructure Status Report

Infrastructure Evolution, Post-Upgrade Runtime Behaviour, Consensus Analysis & Preliminary Mainnet Observation

Prepared by	JERUZSALEM – DAC Infra Tester
Date	Tuesday, 07 July 2026
Environment	Windows + WSL2/Ubuntu dual-node single-machine setup · Indonesia / residential CGNAT
Report Type	Infrastructure Evolution, Runtime Behaviour, and Consensus Analysis Report
Relates to	Report 13 (Cross-Platform Dual Node Synchronization & Peer Connectivity Validation), and Reports 1, 2, 5, 10, 11 of the DAC Testnet (Inception) Technical Report Series
Report Archive	https://github.com/EdLWEISS186/dac-dual-node-cgnat-setup/tree/main/Testnet_(Inception)_Technical_Reports

Executive Positioning: This report consolidates Parts 1–4 of the DAC Testnet Infrastructure Status Report series into a single professional technical submission. It extends prior dual-node validation work by documenting infrastructure evolution, post-upgrade runtime behaviour, consensus and chain-reorganization analysis, cross-platform deterministic execution, and a preliminary Mainnet deployment observation.

Executive Summary

This report documents the current state of the DAC Testnet dual-node deployment following continuous infrastructure evolution and the upgrade of both Windows and Linux (WSL) nodes to gdacnode v1.11.6. Unlike earlier reports that primarily validated synchronization and connectivity in isolation, this report focuses on runtime behaviour, infrastructure maturity, consensus observations, peer management, and cross-platform consistency between the two node deployments.

The investigation traces the environment from its original experimental dual-node design through to a production-like validation platform, examines runtime log behaviour after the v1.11.6 upgrade, analyzes consensus convergence and chain reorganization across both platforms, and concludes with a preliminary Mainnet deployment observation intended to assess architectural reusability beyond the Testnet.

Core Finding: The dual-node, single-machine architecture has matured from an experimental proof-of-concept into a stable, production-like validation platform. Windows and Linux (WSL) nodes converge on identical canonical chain state under identical network conditions, demonstrating deterministic cross-platform execution. Preliminary Mainnet testing indicates the same architecture is technically reusable, though currently constrained by limited Mainnet peer availability.

High-Level Findings

- Infrastructure evolved from a conservative experimental configuration into a production-like platform: Snap synchronization, 50-peer capacity, dedicated per-node RPC/metrics/profiling endpoints, and a 4096 MB execution cache on the Linux node.
- Both nodes maintained healthy synchronization after the gdnacnode v1.11.6 upgrade, with no evidence of persistent synchronization stalls.
- Engine API and Beacon-related runtime messages were observed and assessed as informational client-logging artifacts rather than indicators of Consensus-Layer/Merge-related misconfiguration.
- Reward contract initialization warnings during early sync were confirmed to be expected behaviour that self-resolves once the node reaches the relevant block height.
- Occasional 'block in the future' warnings triggered automatic peer disconnection, with synchronization continuing uninterrupted via alternative peers — evidence of resilient peer management.
- Windows and Linux nodes consistently converged on the same canonical chain despite multiple shallow chain reorganizations, indicating deterministic cross-platform execution.
- `admin.peers` inspection revealed a mixed-client-version environment (pre- and post-v1.11.6 peers) coexisting without loss of protocol interoperability across eth/66, eth/67, eth/68, and snap/1.
- A preliminary Mainnet deployment using the same dual-node topology started successfully, but recurring peer connectivity interruptions were observed during a 2–3 hour window, tentatively attributed to limited active Mainnet peer count.

Balanced Assessment

This report does not identify any critical defect in node software or network consensus. Both nodes handled the v1.11.6 upgrade cleanly, synchronization progressed normally, canonical chain selection was deterministic across platforms, and peer management correctly isolated problematic peers without affecting availability. The Mainnet connectivity interruptions observed are treated as a preliminary, environment-scale observation (small active peer set) rather than a confirmed defect, and Mainnet synchronization was intentionally discontinued for project reasons rather than due to a blocking failure.

Limitations

- This investigation is limited to DAC Testnet. No equivalent long-duration Mainnet dataset was collected; Mainnet observations in this report are preliminary only.
- The Mainnet deployment observation window was approximately 2–3 hours and does not constitute a full synchronization or long-term stability test.
- The author does not have validator-level access, so validator-side clock configuration, block-production scheduling, and internal peer-scoring logic could not be directly inspected.
- No internal DAC protocol documentation was available to the author; interpretations of client log messages (e.g. Engine API, Beacon-related messages) are based on observed runtime behaviour and general Geth-derived client conventions, not on official DAC specifications.
- All findings are based on runtime log behaviour and `admin.peers` snapshots from a single dual-node vantage point (both nodes co-located on one physical machine under the same CGNAT connection); findings may not generalize to nodes in different network conditions.
- Log and screenshot evidence presented in this report reflects representative capture windows rather than continuous, uninterrupted recording across the entire monitoring period.

Background

The project began as an experimental dual-node deployment intended to validate whether two independent DAC nodes could coexist on a single physical machine while remaining fully compatible with the official DAC Testnet. Over time, the environment evolved into a reusable infrastructure platform incorporating separate Windows and Linux (WSL) nodes, dedicated RPC endpoints, metrics collection, runtime profiling, higher peer capacity, and operational monitoring. The environment now serves as the primary foundation for ongoing infrastructure research and future Health Check Toolkit development.

Infrastructure Evolution

Component	Initial Design	Current State	Engineering Impact
Synchronization	Fast/default	Snap synchronization	Faster recovery and deployment
Peer Capacity	Conservative	50 peers	Improved network visibility
RPC	Basic	Dedicated HTTP endpoints per node	Independent monitoring
Metrics	Unavailable	Enabled	Performance monitoring
Profiling	Unavailable	pprof enabled	Runtime diagnostics
Linux Cache	Default	4096 MB	Improved state processing
Architecture	Initial dual-node	Operational dual-node platform	Production-like validation

Environment and Tooling

The current infrastructure consists of two independent DAC Testnet nodes operating simultaneously on a single host. The Windows node and the Linux (WSL) node maintain separate identities, databases, network ports, RPC interfaces, metrics endpoints, and profiling services while participating in the same decentralized network. This topology allows direct comparison of runtime behaviour under identical network conditions. Figure 1 and Figure 2 below reference the current deployment topology and configuration overview used throughout this report.

Figure 1 — Current Infrastructure Topology

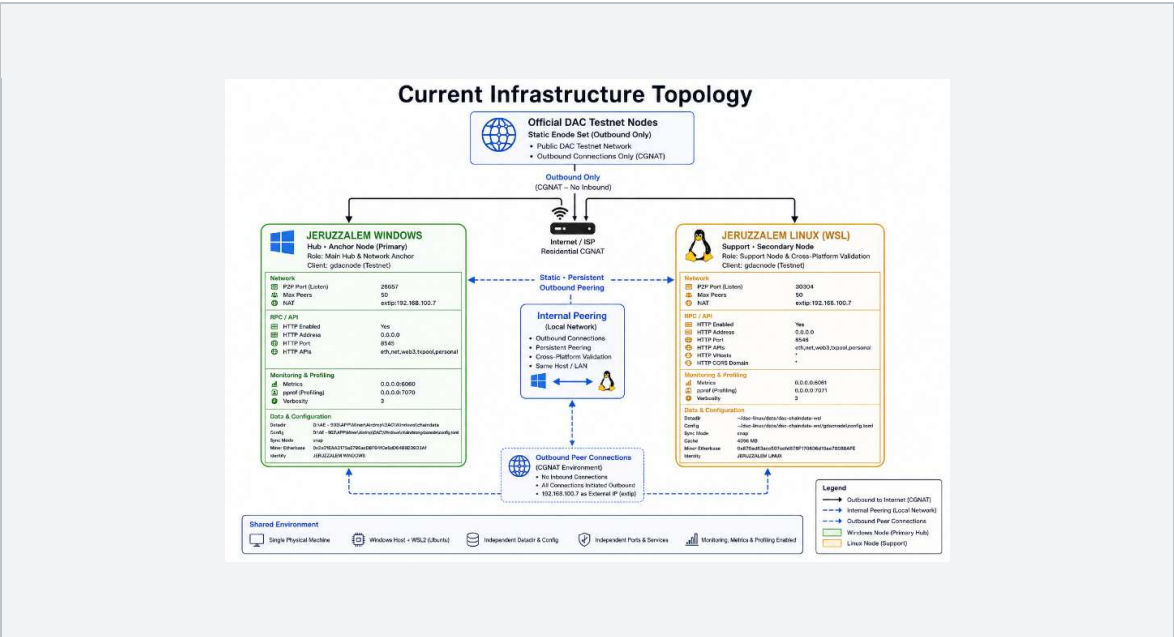
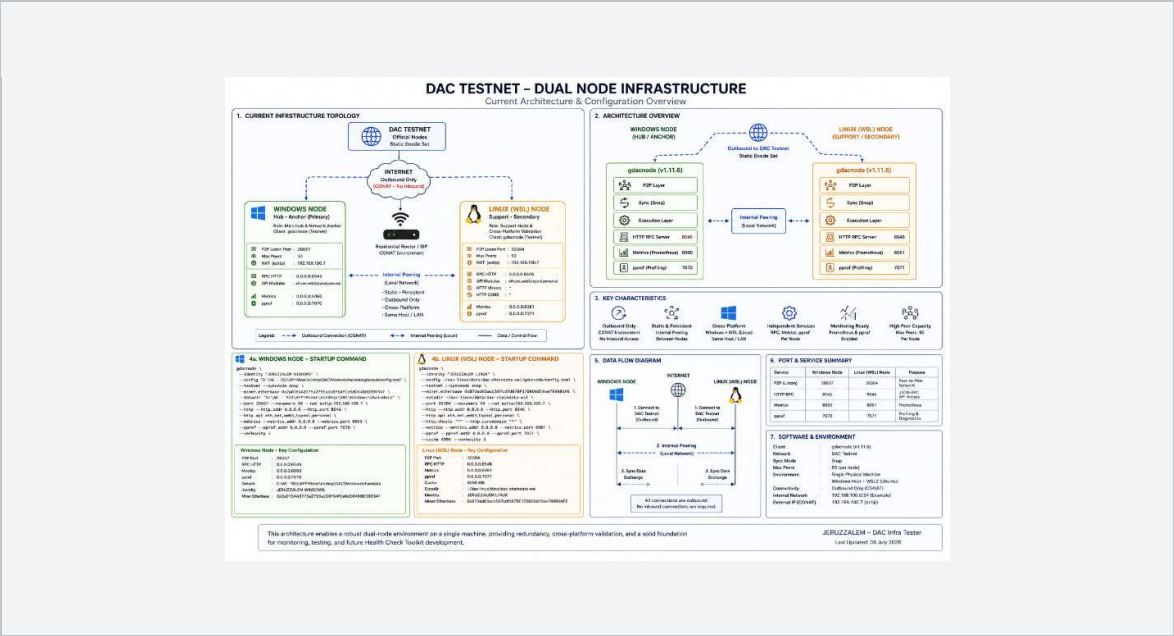


Figure 2 — DAC Testnet Dual-Node Infrastructure: Current Architecture & Configuration Overview



Windows Node

The Windows node operates with Snap synchronization, a dedicated P2P endpoint, HTTP RPC, metrics, pprof profiling, and a peer capacity of fifty connections. Runtime verbosity is configured to collect detailed operational information for behavioural analysis.

Linux (WSL) Node

The Linux node mirrors the Windows deployment while allocating a 4096 MB execution cache to improve state processing performance. Independent RPC, metrics, and profiling endpoints permit simultaneous observation without resource contention.

Parameter	Windows Node	Linux (WSL) Node
Synchronization mode	Snap	Snap
Peer capacity	50	50
RPC	Dedicated HTTP endpoint	Dedicated HTTP endpoint
Metrics	Enabled	Enabled
Profiling	pprof enabled	pprof enabled
Execution cache	Default	4096 MB
Client version	gdacnode v1.11.6	gdacnode v1.11.6

Scope and Methodology

Following the 5 July 2026 upgrade of both nodes to gdacnode v1.11.6, the investigation was structured around continuous runtime observation, log inspection, and admin.peers analysis across four focus areas: post-upgrade synchronization behaviour, consensus and chain-reorganization behaviour, cross-platform deterministic execution and mixed-client interoperability, and a preliminary Mainnet deployment test using the same architecture.

Part	Focus	Primary Validation Method
Part 1	Infrastructure evolution, current deployment architecture	Configuration review, historical comparison
Part 2	Post-upgrade runtime behaviour, synchronization, Engine API/Beacon logging, reward contract init, peer connectivity	Runtime log observation, continuous monitoring
Part 3	Consensus behaviour, chain reorganization, canonical chain selection, mixed-client interoperability	admin.peers inspection, cross-platform log comparison
Part 4	Preliminary Mainnet deployment, infrastructure assessment, recommendations	Controlled Mainnet node start, 2–3 hour observation window

Module Findings

Post-Upgrade Status and Runtime Synchronization Behaviour

On 5 July 2026 both Windows and Linux (WSL) environments were upgraded to the latest available `gdacnode v1.11.6` release. The upgrade was followed by infrastructure validation confirming successful synchronization, stable peer discovery, and continued interoperability between both platforms.

Continuous observation demonstrated that both nodes maintained healthy synchronization after the upgrade. Imported blocks advanced normally without evidence of persistent synchronization stalls. Temporary peer replacement occurred as expected within a decentralized peer discovery environment, while synchronization continued uninterrupted.

Engine API and Beacon-Related Messages

Runtime logs reported that the Engine API was enabled while simultaneously indicating that the chain was not configured for Merge. This behaviour is consistent with modern Geth-derived clients, which include Engine API support regardless of whether the underlying blockchain actively operates a Consensus Layer compatible with Ethereum's Merge architecture. The messages therefore appear informational rather than symptomatic of configuration problems.

```
WARN [07-06|17:57:23.701] Engine API enabled           protocol=eth
WARN [07-06|17:57:23.701] Engine API started but chain not configured for merge yet
```

Several runtime messages referenced Beacon synchronization, including notifications that beacon synchronization had not yet started. Based on observed network behaviour, these messages appear to originate from generic client logging routines rather than indicating that the DAC Testnet itself implements Ethereum Beacon Chain consensus. No operational impact was observed, as synchronization continued normally.

Reward Contract Initialization

During the initial synchronization phase, the execution client reported that the reward contract could not yet be accessed. This condition is expected while the node database has not yet synchronized to the block height at which the contract becomes available. Once synchronization progresses sufficiently, the warning naturally disappears without corrective action.

```
WARN [07-06|17:57:23.830] Served eth_call               reqid=1 duration=19.5092ms
err="execution reverted"
ERROR[07-06|17:57:23.830] Reward contract not found, you have to complete the sync before having
access to the contract.
```

Peer Connectivity and Future Block Detection

Throughout the observation period the peer set remained dynamic. Connections were continuously established and replaced according to normal peer discovery behaviour. Differences of one or more active peers between Windows and Linux nodes were considered expected and do not indicate network partitioning or instability.

Within the captured runtime log window (Windows node, approximately 17:57–18:09), four 'block in the future' events were recorded, each triggering automatic disconnection of the reporting peer. Each event was associated with a distinct peer identity rather than a single repeat offender, indicating the condition is distributed across the peer population rather than caused by one misbehaving node:


```
WARN [07-06|17:58:21.872] Synchronisation failed, dropping peer      peer=38744cc1...b5eb01
err="retrieved hash chain is invalid: block in the future"
WARN [07-06|18:02:21.816] Synchronisation failed, dropping peer      peer=0450f511...2c4a6d
err="retrieved hash chain is invalid: block in the future"
WARN [07-06|18:04:51.448] Synchronisation failed, dropping peer      peer=e52b2299...48295f
err="retrieved hash chain is invalid: block in the future"
```

The node immediately continued synchronization using alternative peers after each disconnection, demonstrating resilient peer management. Several technical hypotheses were considered to explain the recurring pattern; these remain hypotheses rather than confirmed root causes, since validator-side clock configuration and internal peer-scoring logic were not directly observable:

- Validator or peer clock skew — a reporting peer's local clock running ahead of network-median time could cause it to advertise a block timestamped in the future.
- Block propagation latency — under variable network conditions, a block may arrive at a peer before the receiving node's internal time reference considers its timestamp valid.
- Peer selection variance — since each node independently selects peers from a decentralized pool, exposure to a clock-skewed or slow-propagating peer is a matter of which peers happen to be connected at a given moment.
- Mixed client-version influence — differences in timestamp validation tolerance between pre- and post-v1.11.6 clients could marginally change how 'future block' thresholds are enforced.
- Shallow fork frequency — this condition should be more likely to appear during periods of higher fork/reorg activity, since competing blocks are more likely to be produced close to the timestamp validation boundary.

Screenshot 1 — Windows node runtime log: recurring 'beacon sync not yet started' / 'block in the future' pattern across multiple distinct peers

The screenshot displays a Windows command prompt window titled 'Windows Node'. The log shows a continuous stream of messages from a node, primarily consisting of 'Imported new chain segment' and 'Synchronisation failed, dropping peer' messages. The 'Synchronisation failed' messages are the key focus, as they repeatedly state 'err="retrieved hash chain is invalid: block in the future"'. These errors occur for a wide variety of peer IDs, including 38744cc1...b5eb01, 0450f511...2c4a6d, e52b2299...48295f, and many others. The log also shows 'WARN' messages for 'Failed to retrieve beacon bounds for bad block reporting' and 'Failed to retrieve beacon bounds for bad block reporting', which are also associated with the 'block in the future' error. The log is a dense, multi-line text output, with the 'Synchronisation failed' messages appearing at regular intervals, indicating a recurring pattern of synchronization failures.

Consensus Behaviour and Chain Reorganization

The most significant observation during the monitoring period was the remarkable consistency between the Windows and Linux nodes. Despite operating under different operating systems, both clients continuously converged toward the same canonical blockchain. No evidence of execution divergence, inconsistent state transitions, or platform-specific consensus behaviour was identified. This strongly suggests deterministic execution across both environments.

Multiple shallow chain reorganizations were observed during synchronization on both platforms. Within the same captured log window referenced above, the Windows node recorded 1 chain reorganization event while the Linux node recorded 3, across a comparable span of newly imported blocks. This asymmetry is expected: because each node independently receives competing block candidates from its own peer set, the node that happens to receive the eventually-canonical block first will resolve to it directly, while the node that receives a competing block first must perform an explicit reorganization once the canonical block arrives.

Concrete Example — Block 15,137,500 Convergence

A directly comparable example was captured in both node logs around block 15,137,499–15,137,500. Both nodes first imported the same parent block (15,137,499, hash 886fef..f7594f) roughly one second apart. From there, the two nodes took different paths to the same final result:

WINDOWS NODE				
18:02:43.857	Imported	number=15,137,499	hash=886fef..f7594f	
18:02:58.924	Imported	number=15,137,500	hash=7ec408..9897b6	(received first)
18:03:02.934	Chain reorg detected	number=15,137,499	hash=886fef..f7594f	
		drop=1 dropfrom=7ec408..9897b6	add=1 addfrom=493117..88374b	
18:03:02.980	Imported	number=15,137,500	hash=493117..88374b	(canonical, via reorg)
LINUX (WSL) NODE				
18:02:44.894	Imported	number=15,137,499	hash=886fef..f7594f	
18:02:59.709	Imported	number=15,137,500	hash=493117..88374b	(canonical, received first – no reorg needed)
18:02:59.829	Imported	number=15,137,500	hash=7ec408..9897b6	(competing block, correctly not adopted)

Both nodes reached the identical canonical hash for block 15,137,500 (493117..88374b). The Linux node received the eventually-canonical block directly on first attempt and required no reorganization, converging at 18:02:59.709. The Windows node received the competing block first and required an explicit reorganization roughly 3.27 seconds later, converging at 18:03:02.980. This is direct log evidence — rather than an inferred assumption — that the two platforms execute identical consensus and canonical-chain-selection logic, differing only in the order in which network propagation delivered competing blocks.

[illegible][illegible]

Canonical Chain Selection and Cross-Platform Deterministic Execution

Both nodes consistently selected identical canonical chains following temporary forks, as demonstrated concretely in the block 15,137,500 example above. Even when block competition occurred, the execution clients independently reached the same final chain without manual intervention, confirming that chain selection rules remain deterministic across platforms rather than merely appearing consistent by coincidence.

One objective of the dual-node architecture was to determine whether Windows and Linux produced identical blockchain behaviour when exposed to the same network conditions. Runtime observations, supported by matching canonical hashes captured directly in both logs, indicate that imported blocks, synchronization progress, peer discovery, and reorganization events remained functionally equivalent, with no platform-specific execution anomalies detected.

Peer Management Strategy and Mixed Client Version Environment

The execution client demonstrated resilient peer management throughout the observation period. Peers transmitting invalid or future-dated block chains were automatically disconnected while synchronization continued using alternative peers, as evidenced above. This behaviour illustrates that peer validation operates independently from consensus progression, preventing problematic peers from degrading node availability.

Inspection of `admin.peers` revealed that the DAC Testnet currently operates in a mixed-version environment. Several peers had already upgraded to `gdacnode v1.11.6` while others continued operating previous releases. No official network-wide upgrade announcement was observed by the author during this period, suggesting that client adoption occurred gradually. Despite version diversity, protocol interoperability remained functional and no incompatibility affecting synchronization was observed.

Protocol Negotiation

Observed peer sessions negotiated multiple protocol versions, including `eth/66`, `eth/67`, `eth/68`, and `snap/1`. The coexistence of several protocol revisions indicates backward compatibility between participating clients and supports stable communication across mixed software versions.

Preliminary Mainnet Deployment Observation

Following the successful maturation of the DAC Testnet infrastructure, an initial attempt was made to deploy both Windows and Linux (WSL) Mainnet nodes using the same dual-node, single-machine topology previously validated on the Testnet. The objective was to verify whether the architecture could be reused without significant modification under Mainnet conditions.

The website's `enode-list` subdomain was updated from the Testnet endpoint to the Mainnet endpoint, and the Mainnet `enode` list was confirmed to be available. Both Mainnet nodes were started successfully, although synchronization was intentionally not allowed to complete.

During an observation period of approximately two to three hours, recurring peer connectivity interruptions were noted. A possible explanation is the relatively small number of currently active Mainnet nodes, which may reduce peer diversity and connection stability. This interpretation should be considered a technical hypothesis rather than a confirmed conclusion, consistent with the Limitations noted earlier in this report.

Mainnet synchronization was intentionally discontinued because the ongoing infrastructure project and related tooling currently depend on maintaining the Testnet environment online for continuous development and validation.

Technical Analysis

The combined observations indicate that the execution client successfully distinguishes between informational runtime messages, temporary synchronization conditions, and events requiring defensive peer management. Importantly, synchronization proceeds after peer replacement, suggesting that peer validation mechanisms operate independently from block execution and do not compromise overall network participation.

Observation Class	Interpretation	Operational Impact
Engine API / Beacon logging	Informational, generic Geth-derived client logging; not indicative of Merge/Consensus-Layer misconfiguration	None
Reward contract init warning	Expected during early sync before relevant block height is reached	Self-resolving
'Block in the future' + peer disconnect	Distributed across multiple distinct peers; hypothesized clock skew, propagation delay, peer selection variance, or mixed-version timestamp tolerance	None — synchronization continues via alternative peers
Shallow chain reorganization	Normal consensus convergence under competing validators; asymmetric reorg counts between nodes reflect propagation order, not divergence	None
Block 15,137,500 cross-node convergence	Confirmed via matching canonical hash in both node logs; different arrival order (Linux direct, Windows via reorg), same final state ~3.27s apart	None — demonstrates deterministic execution
Mixed client version peers (pre-/post-v1.11.6)	Gradual, non-coordinated network-wide adoption	None — protocol negotiation remains backward-compatible
Mainnet peer connectivity interruption	Hypothesized limited active Mainnet peer diversity	Blocks full Mainnet sync validation until more nodes are active

Engineering Assessment

From an engineering perspective, the runtime behaviour indicates a healthy decentralized network. Temporary forks, shallow reorganizations, peer replacement, and dynamic peer discovery are expected characteristics of active blockchain environments. More importantly, the network consistently converged toward a common canonical chain — directly confirmed through matching block hashes captured in both node logs — demonstrating that consensus remained stable throughout the observation period. The observations also reinforce confidence that Windows and Linux deployments can be operated simultaneously without introducing consensus inconsistencies.

Infrastructure Assessment

The current infrastructure demonstrates a significant evolution from the original experimental setup into a mature multi-platform validation environment. Independent Windows and Linux nodes operate reliably, expose monitoring interfaces, and provide an effective platform for studying synchronization, consensus behaviour, peer discovery, and protocol interoperability.

Engineering Recommendations

- Publish network-wide client upgrade announcements whenever new node releases become available.
- Provide a minimum supported client version policy for operators.
- Monitor validator and peer clock synchronization to reduce future-block events; consider NTP-based diagnostics as part of the planned Health Check Toolkit.
- Continue collecting long-duration runtime logs for behavioural trend analysis, including reorg frequency and future-block event distribution across peers.
- Integrate synchronization, peer-count, protocol-version, and runtime diagnostics into the planned Health Check Toolkit.
- Perform a dedicated Mainnet connectivity study after additional Mainnet nodes become available.

Future Work

Future development will focus on automated infrastructure monitoring, cross-node health validation, runtime performance visualization, and comparative studies between DAC Testnet and Mainnet network behaviour.

Conclusion

The observations presented throughout this report indicate that the current DAC Testnet deployment is operationally healthy. Cross-platform deterministic execution was consistently observed and, in the case of block 15,137,500, directly confirmed through matching canonical hashes captured in both nodes' runtime logs. Shallow chain reorganizations converged correctly toward a canonical chain, peer replacement occurred without interrupting synchronization, and mixed client versions continued to interoperate successfully. The dual-node architecture has therefore proven to be a reliable platform for ongoing infrastructure research and future tooling development.

Final Engineering Summary: The dual-node, single-machine architecture has evolved from an experimental proof-of-concept into a stable, production-like validation platform. Post-upgrade runtime behaviour, consensus convergence, and cross-platform execution all point to a healthy, deterministic network — supported by direct log evidence rather than inference alone. The preliminary Mainnet observation confirms architectural reusability, with peer connectivity constraints attributed to the current scale of Mainnet adoption rather than to any defect in the dual-node design itself.

References

***Background Technical References (non-normative for DAC):** DAC Quantum Chain Testnet is EVM-compatible at the execution layer, but operates its own independent node client (gdacnode), network specification, and consensus configuration distinct from Ethereum Mainnet. The following references describe general behaviour of Geth-derived execution clients and related peer-to-peer/consensus concepts that inform the interpretations in this report (e.g. Engine API messages, Snap sync, protocol negotiation). They are cited as background context for client-level behaviour only, not as authoritative documentation of DAC's own protocol.*

- Ethereum Yellow Paper — formal specification of the Ethereum Virtual Machine and state-transition model referenced by many EVM-compatible execution clients.
- Go Ethereum (Geth) Documentation — official documentation for the execution client family from which gdacnode is derived, covering synchronization modes, RPC, and logging conventions.

- Engine API Specification — describes the Engine API interface used for Execution Layer / Consensus Layer communication in post-Merge Ethereum-derived clients.
- Snap Sync Documentation — describes the Snap synchronization protocol used by both nodes in this report's configuration.
- libp2p Documentation — general peer-to-peer networking library concepts relevant to peer discovery and connection management patterns observed in this report.
- Ethereum Consensus Specifications — reference specifications for Beacon Chain / consensus-layer terminology referenced in client log messages.

Part Coverage Map

Part	Included in Final Report Section
Part 1	Background, Infrastructure Evolution, Environment and Tooling (Windows/Linux Node), Figures 1–2
Part 2	Post-Upgrade Status & Runtime Sync, Engine API/Beacon, Reward Contract Init, Peer Connectivity/Future Block Detection
Part 3	Consensus Behaviour, Canonical Chain Selection, Peer Management/Mixed Client Versions, Protocol Negotiation
Part 4	Preliminary Mainnet Observation, Technical Analysis, Infrastructure Assessment, Recommendations, Future Work, Conclusion

Evidence Index

Evidence	Description
Figure 1	Current Infrastructure Topology diagram (to be inserted)
Figure 2	Dual-Node Infrastructure: Architecture & Configuration Overview diagram (to be inserted)
Screenshot 1	Windows node log — recurring 'block in the future' pattern across distinct peers (to be inserted)
Screenshot 2	Linux (WSL) node log — Imported → Chain reorg detected → Imported sequences (to be inserted)
Screenshot 3	Windows node log — same session as Screenshot 2, for cross-platform comparison (to be inserted)
Block 15,137,500 convergence	Canonical hash 493117..88374b confirmed identical in both Windows and Linux node logs; Windows via reorg at 18:03:02.980, Linux via direct import at 18:02:59.709

Repository & Documentation

Project Repository: <https://github.com/EdLWEISS186/dac-dual-node-cgnat-setup>

Complete Technical Report Archive: [https://github.com/EdLWEISS186/dac-dual-node-cgnat-setup/tree/main/Testnet_\(Inception\)_Technical_Reports](https://github.com/EdLWEISS186/dac-dual-node-cgnat-setup/tree/main/Testnet_(Inception)_Technical_Reports)